Examen final — Fase 3: CORS, 401, filtros del listado, H-5 y contrato de la API

Rama: `final-fase-3-cierre-backend` (desde `main`, `40b2674`) · **Fecha:** 12/11/2026 · **Estado:** implementada y probada; sin commit **Por qué existe esta fase:** el frontend va a llamar a la API desde un navegador. Antes de escribir una sola pantalla había que quitar lo que se lo impedía y fijar el contrato. **Consigna (PDF, examen final, punto 4b):** el monitoreo debe tener «filtros y ordenamiento por estado u otros criterios».

1. Qué se hizo

#	Cambio	Problema que resuelve
1	**CORS dentro de la cadena de seguridad** (`http.cors()` + `CorsConfigurationSource`)	H-30: el preflight `OPTIONS` respondía **401** (medido en local y en la VM) y el navegador bloqueaba toda llamada entre orígenes
2	**401 propio**: JSON estándar y **sin** `WWW-Authenticate`; se quita `httpBasic`	El 401 traía `WWW-Authenticate: Basic`: al vencer el token el navegador abría su diálogo de usuario y contraseña. Además `httpBasic` era un camino de autenticación alterno (usuario y contraseña en cada pedido) que nadie usaba
3	`GET /orders?status=&sort=`: filtro por estado y orden elegible	Consigna 4b. Antes el orden era fijo y no había filtro
4	**H-5**: clave JWT (`JWT_SECRET`) y orígenes CORS (`CORS_ALLOWED_ORIGINS`) por variables de entorno	La clave estaba escrita en el código (`MyVerySecretKey`) y el CORS abierto. Sin definirlas rige lo anterior: el despliegue actual no cambia
5	**Contrato de la API congelado** (`docs/contrato-api.md`)	El frontend heredado se había hecho contra una API imaginada; ahora se conecta una sola vez a un contrato verificado

2. Decisiones de diseño

#	Decisión	Por qué
S-1	CORS por defecto **abierto** (`*`), configurable con `CORS_ALLOWED_ORIGINS`	Valor actual como respaldo (aprobado): no rompe el despliegue. Es seguro frente a CSRF porque el token viaja en un header y no en una cookie. En producción conviene limitarlo
S-2	Métodos `GET, POST, PUT, DELETE, OPTIONS` y cabeceras `Authorization, Content-Type, Accept`	Lista cerrada, la que la API usa. Cualquier otra se rechaza (403)
S-3	Se expone el header `Location`	El alta de órdenes (SAP) responde 201 con ese header
S-4	El 401 lleva las cabeceras CORS	Si no, el navegador oculta el 401 y el frontend no puede cerrar la sesión
S-5	Se quita `httpBasic` y se define un `AuthenticationEntryPoint` propio	Sin `httpBasic` Spring respondería 403 a los anónimos; el punto de entrada explícito mantiene el 401
S-6	`ErrorResponseWriter` común para el 401 y el 403	Un solo lugar arma el JSON estándar desde los filtros (no hay `ObjectMapper` y Spring Boot 4 trae dos Jackson)
S-7	`status` es el **número** 1 a 4 (como en el JSON)	Coherente con `"status": 2`; cualquier otro valor da 400 con `message`
S-8	`sort` con **lista cerrada** de campos: `orderNumber`, `plannedLoadDateTime`, `receptionDateTime`, `status`, `preset`	El cliente no puede ordenar por columnas internas (por ejemplo `activationPassword`, que es secreta). Cualquier otro campo da 400 con la lista de permitidos
S-9	El número de orden descendente **desempata**	Con valores repetidos (muchas órdenes con el mismo estado) la paginación saltaría o repetiría filas
S-10	`JwtSecret` es el único lugar que resuelve la clave	El login firma y el filtro verifica con la misma clave; sin definirla, los mismos bytes que antes (los tokens vigentes siguen válidos)

#	Decisión	Por qué
S-11	Cambiar `JWT_SECRET` invalida los tokens vigentes	Es el efecto buscado; se documenta para el despliegue
S-12	El contrato se **genera** desde `/v3/api-docs` y respuestas reales, y el generador comprueba que cubre todas las rutas	Un contrato escrito a mano se desfasa (justo lo que le pasó al frontend heredado)

3. Inventario de archivos

Archivo	Acción
`security/SecurityConfiguration`	Reescrita: `http.cors`, `CorsConfigurationSource`, punto de entrada de 401, sin `httpBasic`, filtro con `JwtSecret`
`security/RestAuthenticationEntryPoint`, `security/ErrorResponseWriter`	**Nuevos** (el 403 de `RestAccessDeniedHandler` ahora usa el mismo escritor)
`auth/filters/JwtSecret`	**Nuevo**; `JWTAuthorizationFilter` y `AuthRestController` lo usan en lugar de `AuthConstants.SECRET`
`controllers/OrderRestController`, `business/IOrderBusiness`, `OrderBusiness`, `model/persistence/OrderRepository`	Listado con `status` y `sort`: `search(status, pageable)` reemplaza a `findAllByOrderByOrderNumberDesc`
`application.properties`	`jwt.secret` y `cors.allowed-origins` por variable de entorno
Tests: `CorsAndAuthErrorsTest`, `CorsRestrictedOriginsTest`, `JwtSecretTest`, `OrderListQueryTest`, `TestTokens`; `RolesAccessTest` ajustado	**Nuevos** / modificado
`docs/postman/…collection.json`	Carpeta `10 - Cierre del backend: CORS, 401 y filtros`
`docs/contrato-api.md`	**Nuevo**, generado

4. Recorrido del preflight (lo que antes fallaba)

1. El navegador, desde `http://localhost:3000`, va a hacer `GET /api/v1/orders` con `Authorization`. Como el header dispara CORS, primero manda `OPTIONS /api/v1/orders` con `Origin`, `Access-Control-Request-Method: GET` y `Access-Control-Request-Headers: authorization`. **No lleva token.**
2. La cadena de seguridad ahora tiene `CorsFilter` **antes** de la autorización: reconoce el preflight, lo compara con la configuración y responde `200` con `Access-Control-Allow-*`. La petición ni siquiera llega a `anyRequest().authenticated()`.
3. Recién ahí el navegador manda el `GET` real, con el token. Si el token falta o venció, responde `401` **con** las cabeceras CORS (el frontend puede leerlo) y sin `WWW-Authenticate` (no hay diálogo).

Antes el `CorsFilter` no estaba en la cadena (la configuración era un mapeo de Spring MVC, que corre después de la autorización): el preflight caía en el 401.

5. Pruebas ejecutadas (12/11/2026)

Prueba	Resultado
`./mvnw -B test`	**248 tests, 0 fallas** (215 anteriores + 33 nuevos)
Pruebas de mutación manuales: sin `http.cors()`, el `sort` acepta cualquier campo, la clave configurada se ignora, `httpBasic` vuelto a habilitar	Las 4 detectadas. **La de `httpBasic` no lo estaba al principio**: reveló que faltaba probar que usuario y contraseña por Basic ya no autentican; se agregó el test y quedó detectada

Prueba	Resultado
Escenario real con MySQL, configuración por defecto (`curl`)	**26 verificaciones, 0 incorrectas**: preflight 200 (antes 401), 401 sin `WWW-Authenticate`, token vencido, Basic rechazado, y filtros y orden **comparados contra la base** (los 4 estados, el total, `ORDER BY` por fecha de carga, paginación sin repetidos)
Escenario real con `JWT_SECRET` y `CORS_ALLOWED_ORIGINS` definidos	**8 verificaciones, 0 incorrectas**: la clave por defecto deja de servir, el origen configurado se devuelve y uno ajeno recibe 403
Chrome real (headless) con una página en `http://localhost:3000`	Desde el origen **permitido**: login, `GET` con `Authorization`, lectura del 401 con su JSON y `PUT` con JSON, todo funciona. Desde `http://localhost:3001` (no permitido): el navegador bloquea las 5 llamadas
Postman con newman, colección completa	**246 pedidos, 539 aserciones, 0 fallas** (línea base de la Fase 2: 219 y 483; la carpeta 10 agrega 27 y 56)
Contrato	17 operaciones publicadas = 17 documentadas (el generador falla si difieren); sin tokens ni trazas incrustados

La prueba con Chrome es la única que reproduce lo que hace el frontend: `curl` y Postman no aplican la política de mismo origen, por eso el problema H-30 pasó inadvertido hasta que se midió el preflight a mano.

6. Cómo configurar el despliegue (paso manual, con autorización; no se hizo)

Variables de entorno del servicio `iw3-app` en `~/infra`:

```
JWT_SECRET=<cadena aleatoria de 32 o más caracteres>
CORS_ALLOWED_ORIGINS=http://143.0.100.187:3000,https://jc755.duckdns.org
```

- Sin ellas todo queda como hoy (clave del repositorio y CORS abierto), y el log avisa «se usa la clave por defecto».
- Al definir `JWT_SECRET` **todos los usuarios tienen que volver a iniciar sesión** (los tokens viejos dejan de valer).
- `CORS_ALLOWED_ORIGINS` debe incluir el origen exacto del frontend (esquema, host y puerto).

7. Limitaciones

- El CORS sigue abierto por defecto (S-1): hasta que se defina la variable en la VM, cualquier sitio puede llamar a la API desde un navegador (necesita igualmente un token válido).
- La clave JWT por defecto sigue en el repositorio hasta que se defina `JWT_SECRET` en la VM.
- Sigue sin haber límite de intentos de login (H-17), *refresh token* ni *logout* en el servidor; el token dura 1 hora (H-18).
- `status` y `sort` solo existen en `GET /orders`; no hay búsqueda de texto (por patente, cliente, etc.). El frontend puede filtrar eso en la página que tenga cargada.
- Un `{orderNumber}` no numérico o un `size` no numérico responden 400 con el formato de Spring, sin `message` (H-23).
- Las cadenas de error de `devInfo` en desarrollo traen el *stack trace*; en producción es `null`.

8. Guía de defensa

1. **¿Qué es CORS y por qué fallaba?** Es la política del navegador que impide llamar a otro origen salvo que el servidor lo autorice. Para pedidos con `Authorization` el navegador manda antes un `OPTIONS` (preflight) sin token. La API lo respondía 401 porque el filtro de CORS no estaba en la cadena de seguridad.
2. **¿Por qué `curl` y Postman no lo detectaban?** No son navegadores: no hacen preflight ni aplican la política de mismo origen.

3. **¿Qué diferencia hay entre `addCorsMappings` y `http.cors()`?** El primero configura Spring MVC, que corre después de la autorización; el segundo agrega el `CorsFilter` a la cadena de seguridad, antes de ella.
4. **¿Por qué el 401 también lleva las cabeceras CORS?** Si no las tuviera, el navegador ocultaría la respuesta y el frontend no podría saber que el token venció.
5. **¿Qué era `WWW-Authenticate: Basic` y qué provocaba?** Un desafío HTTP Basic: el navegador abre su diálogo nativo de usuario y contraseña. Con el token vencido el usuario veía ese diálogo en lugar de volver al login.
6. **¿Por qué se quitó `httpBasic`?** La API solo usa JWT. `httpBasic` permitía además autenticarse con usuario y contraseña en cada pedido, un camino no previsto.
7. **¿Qué pasaría si solo se quitara `httpBasic`?** Spring respondería 403 a los anónimos; por eso se define un `AuthenticationEntryPoint` propio que devuelve el 401 con el JSON estándar.
8. **¿Cómo funciona el filtro por estado y el orden?** `GET /orders?status=2&sort=plannedLoadDateTime,desc`: el controller valida `status` (1 a 4) y el `sort` contra una lista cerrada y arma un `Pageable`; el repositorio aplica `where (:status is null or o.status = :status)` y el orden.
9. **¿Por qué una lista cerrada de campos para ordenar?** Para que el cliente no ordene por columnas internas (la contraseña de activación) ni provoque errores con campos inexistentes.
10. **¿Para qué el desempate por número de orden?** Con muchos registros iguales en el campo elegido el orden no es determinista y la paginación puede repetir o saltear filas.
11. **¿Qué es H-5 y qué se hizo?** La clave JWT estaba en el código y el CORS abierto. Ahora salen de `JWT_SECRET` y `CORS_ALLOWED_ORIGINS`, con el valor anterior como respaldo para no romper el despliegue.
12. **¿Qué pasa con los tokens al cambiar `JWT_SECRET`?** Dejan de ser válidos: todos los usuarios deben volver a iniciar sesión.
13. **¿Por qué el contrato se genera y no se escribe a mano?** Un contrato manual se desfasa del código; el generador lee el OpenAPI real y falla si hay rutas sin documentar.
14. **¿Cómo se probó que funciona en un navegador?** Con una página servida desde `http://localhost:3000` y Chrome headless: desde el origen permitido todo funciona, desde otro el navegador bloquea las llamadas.

9. Lista de comprobación para rendir

- [ ] Explico el recorrido del preflight de la sección 4 y qué cambió en la cadena de seguridad.
- [ ] Sé por qué `curl` no detectaba el problema y qué prueba sí lo reproduce.
- [ ] Puedo decir qué hacen `JWT_SECRET` y `CORS_ALLOWED_ORIGINS` y qué ocurre si no se definen.
- [ ] Explico la lista cerrada de `sort` y el desempate.
- [ ] Sé qué contiene el contrato y para qué se congela.